

FastAPI的介绍



FastAPI 简介

FastAPI 是一个现代、高性能的 Python Web 框架，用于构建 API。它基于 **Starlette**（异步 Web 框架）和 **Pydantic**（数据验证库），结合了异步编程和类型提示，兼顾开发效率与运行性能。

文档： <https://fastapi.tiangolo.com>

源码： <https://github.com/tiangolo/fastapi>

FastAPI 的核心优势

高性能：

- FastAPI 基于异步 I/O，性能接近 Node.js 和 Go。
- 使用 **Uvicorn**（ASGI 服务器），支持高并发请求。
- 性能对比（基于 TechEmpower 等基准测试，简化为每秒请求数）：
 - **FastAPI**：~3000 请求/秒（异步，轻量）
 - **Flask**：~1000 请求/秒（同步，受 WSGI 限制）
 - **Django**：~800 请求/秒（同步，ORM 和中间件开销较大）

类型提示提升开发效率：

- FastAPI 使用 Python 类型提示（通过 Pydantic）进行数据验证，减少手动校验代码。

- 类型提示使代码更易读，IDE（如 VSCode）提供自动补全和错误提示。
- 示例：定义一个带类型提示的 API 端点：

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3
4 app = FastAPI()
5
6 class Item(BaseModel):
7     name: str
8     price: float
9     is_offer: bool = None
10
11 @app.post("/items/")
12 async def create_item(item: Item):
13     return item
```

提示

上述代码自动验证请求体中的 name（字符串）、price（浮点数）和 is_offer（布尔值，可选），无需手动解析 JSON。

自动生成 API 文档：

- FastAPI 内置 **Swagger UI** 和 **ReDoc**，自动生成交互式 API 文档。
- 开发者只需编写代码，文档即自动生成，减少维护成本。

异步支持：

- 支持 `async/await` 语法，适合高并发场景（如实时聊天、流处理）。
- 比传统同步框架（如 Flask）更适合现代 Web 应用。

为什么选择 FastAPI?

- **开发速度快**: 类型提示和自动文档减少重复工作。
- **性能优异**: 异步架构支持高并发, 适合生产环境。
- **社区活跃**: 快速增长的生态, 兼容 Starlette 和 Pydantic 的扩展。
- **易于上手**: Python 开发者只需掌握基本类型提示即可快速构建 API。

CGI/WSGI/ASGI了解



- CGI 是最早的通用接口, 解决服务器与动态内容生成程序的通信问题, 但性能低下。
- WSGI 针对 Python 生态优化, 取代 CGI, 成为 Python Web 开发的主流标准, 专注于同步 Web 应用。
- ASGI 是 WSGI 的升级, 适应异步编程和现代 Web 需求 (如 WebSocket、HTTP/2), 兼容 WSGI 应用。

实时效果反馈

1. FastAPI 的核心性能优势主要得益于以下哪项?

- A** 基于同步 I/O 的 Flask 框架
- B** 异步 I/O 和 Uvicorn ASGI 服务器
- C** Django 的 ORM 优化
- D** CGI 通用接口

答案

FastAPI环境搭建



创建虚拟环境

虚拟环境隔离项目依赖，避免全局环境冲突。以下使用conda工具：

- 1 创建虚拟环境：

```
1 | conda create -n fastapi_env python=3.12
```

- 2 激活虚拟环境：

```
1 | conda activate fastapi_env
```

- 3 退出虚拟环境

```
1 | conda deactivate
```

安装 FastAPI 及其依赖

- 1 在虚拟环境中安装 FastAPI：

```
1 | pip install "fastapi[standard]"
```

指定版本安装：

```
1 pip install fastapi==0.115.12
2 pip install uvicorn==0.34.2
```

FastAPI第一个 API与启动项目



```
1 from fastapi import FastAPI
2 import uvicorn
3
4 app = FastAPI()
5
6 @app.get("/")
7 def read_root():
8     return {"Hello": "world"}
9
10 if __name__ == '__main__':
11
12     uvicorn.run('main01:app', host='0.0.0.0', port
13               =8000, reload=True, debug=True)
14     # uvicorn file_name:object --reload
```

启动项目

① 运行python脚本

```
1 uvicorn.run('main01:app', host='0.0.0.0', port=8000, reload=True)
2 uvicorn.run(app, host='0.0.0.0', port=8000, reload=True, debug=True)
```

代码中要用服务器启动代码

② 服务器启动项目

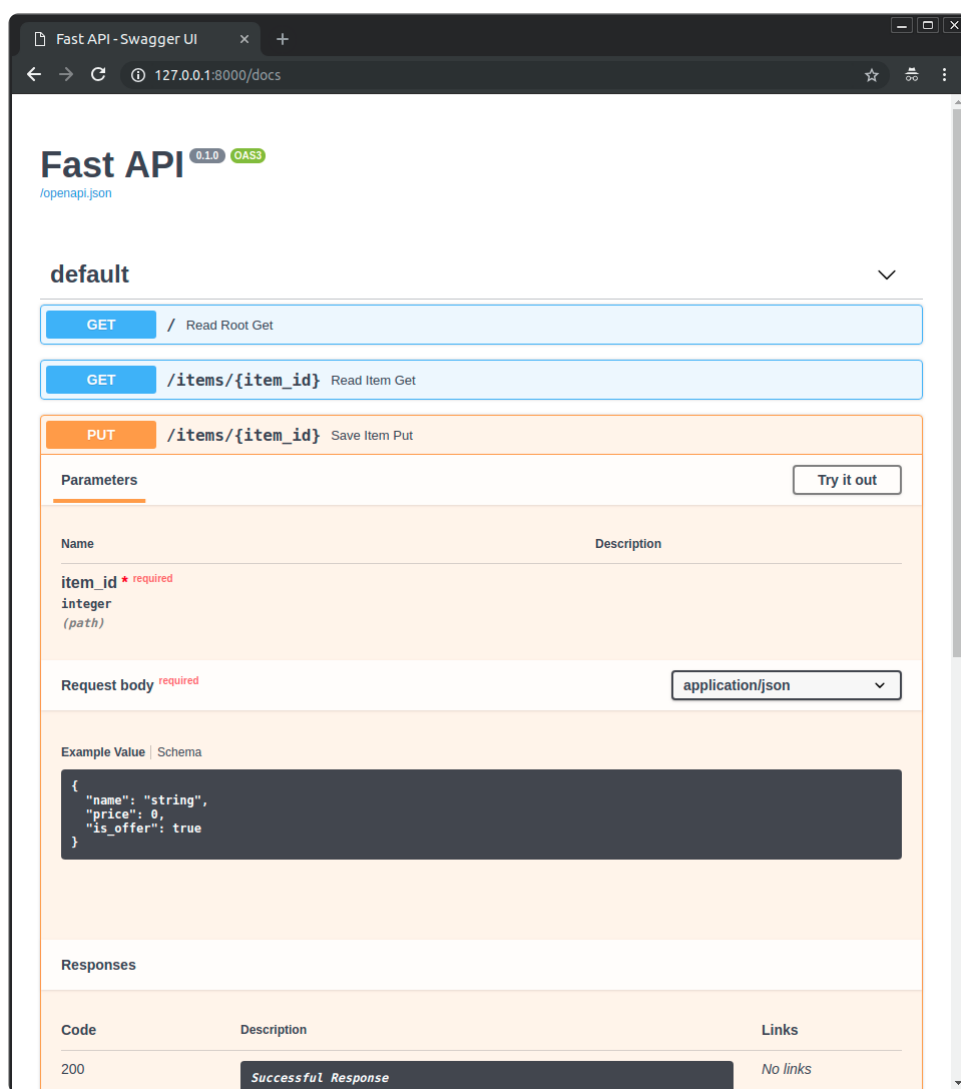
```
1 uvicorn file_name:object --reload
```

③ 调试命令

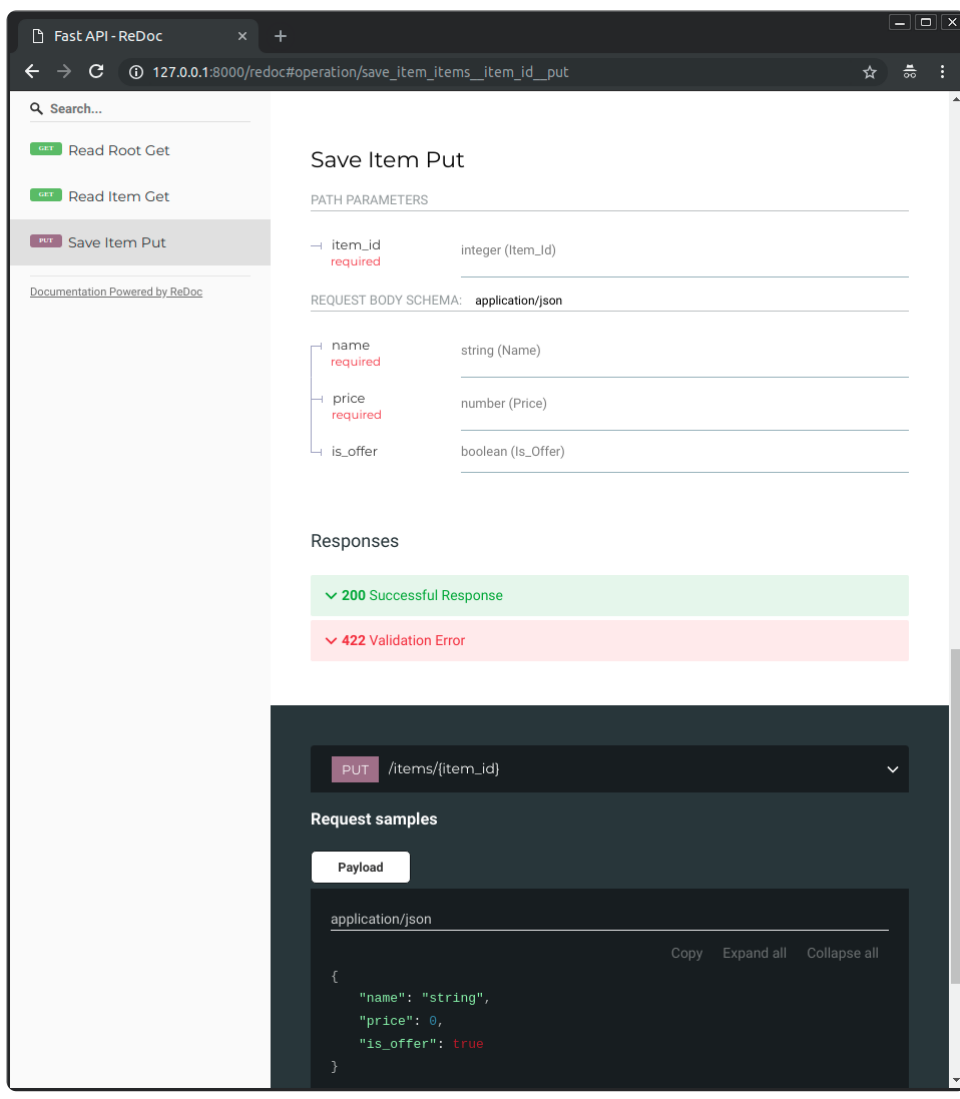
```
1 fastapi dev filename.py
```

访问项目

<http://127.0.0.1:8000/docs>



<http://127.0.0.1:8000/redoc>



用AI生成API接口



使用 AI 生成 FastAPI 代码

AI 工具可以通过自然语言提示生成代码。我们将模拟使用 AI 生成一个简单的 FastAPI 应用。

- deepseek: <https://chat.deepseek.com/>
- 豆包: <https://www.doubao.com/chat/>
- 通义: <https://www.tongyi.com/qianwen/>
- Kimi: <https://kimi.moonshot.cn/>
- 等等

提示示例

向 AI 提供以下提示：

```
1 # 编写一个FastAPI的HelloWorld程序
2
3 # 请为我生成一个 FastAPI 应用程序，功能：返回欢迎消息。
```

以下生成的代码

```
1 from fastapi import FastAPI
2 import uvicorn
3
4 app = FastAPI()
5
6 @app.get("/")
7 def read_root():
8     return {"Hello": "world"}
```

注意事项

AI 生成的代码可能存在小错误（如缺少字段约束），需手动检查

AI的生成的问题

① AI生成代码的局限性

- **缺乏上下文理解**：AI生成的代码可能符合语法，但未必适配实际业务逻辑（如身份验证流程、数据库设计）。
- **难以维护与调试**：若学生只会“复制粘贴”，遇到错误或需求变更时将束手无策。
- **部署与运维盲区**：AI通常不涉及服务器配置、性能优化、监控等生产环境关键环节。

② 核心能力的不可替代性

- **架构设计思维**：如何划分模块、设计REST API、管理依赖关系，需系统性训练。
- **调试与问题定位**：理解上下文机制、请求生命周期，才能快速排查异常。
- **安全与性能意识**：防止SQL注入、XSS攻击，或优化数据库查询，需人工介入设计。

FastAPI路径参数



FastAPI 支持使用 Python 字符串格式化语法声明**路径参数**（变量）：

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/args1/1")
6 def path_args1():
7     return {"message": "id1"}
```

```
9 @app.get("/args2/{id}")
10 def path_args2():
11     return {"message": "id2"}
12
13 @app.get("/args3/{id}")
14 def path_args3(id):
15     '''函数的顺序就是路由的顺序'''
16     return {"message": id}
17
18 @app.get("/args4/{id}")
19 def path_args4(id: int):
20     '''函数的顺序就是路由的顺序'''
21     return {"message": id}
22
23 @app.get("/args5/{id}/{name}")
24 def path_args5(id: str, name: str):
25     '''函数的顺序就是路由的顺序'''
26     return {"id": id, "name": name}
27
28
29 if __name__ == "__main__":
30     import uvicorn
31     uvicorn.run('aaa:app', host="127.0.0.1",
port=8000, reload=True)
```

实时效果反馈

1. 在FastAPI中，关于路径参数的说法哪个是正确的？

- ☒ A 路径参数只能是字符串类型
- ☐ B 路径参数可以自动转换为声明的类型

- C 路径参数必须提供默认值
- D 路径参数不能包含特殊字符

答案

1=>B

FastAPI查询参数



声明的参数不是路径参数时，路径操作函数会把该参数自动解释为**查询参数**

查询字符串是键值对的集合，这些键值对位于 URL 的 ? 之后，以 & 分隔

例如：

```
1 http://127.0.0.1:8000/items/?page=1&limit=10
```

```
1 from fastapi import FastAPI
```

```
2
3 app = FastAPI()
4
5 @app.get("/query1")
6 def page_limit(page, limit):
7     return {"page": page, "limit": limit}
8
9 @app.get("/query2")
10 def page_limit2(page, limit=None):
11     if limit:
12         return {"page": page, "limit":
13 limit}
14     return {"page": page}
15
16 @app.get("/query3")
17 def page_limit3(page, limit, info: int):
18     return {"page": page, "limit": limit,
19 "info": info}
20
21 @app.get("/query4/{page}")
22 def page_limit4(page, limit, info: int):
23     return {"page": page, "limit": limit,
24 "info": info}
25
26 if __name__ == "__main__":
27     import uvicorn
28     uvicorn.run('aaa:app', host="127.0.0.1",
29 port=8000, reload=True)
```

实时效果反馈

1. 在 FastAPI 中，如果一个查询参数没有设置默认值，它的行为是什么？

- A 自动赋值为 None
- B 是必填参数，缺少时会返回错误
- C 自动赋值为 0
- D 会被忽略，不影响函数执行

答案

1=>B

FastAPI请求体



FastAPI 使用**请求体**从客户端（例如浏览器）向 API 发送数据。

请求体是客户端发送给 API 的数据。

发送数据使用 POST（最常用）、PUT、DELETE、PATCH 等操作。

定义模型

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3
4 class Item(BaseModel):
5     name: str
6     description: str | None = None
7     price: float
8
9 app = FastAPI()
10
11 @app.post("/items/")
12 async def create_item(item: Item):
13     return item
```

实时效果反馈

1. 在 FastAPI 中，请求体通常以什么格式传递给 API 端点？

- ☒ A URL 查询字符串（如 ?key=value）
- ☐ B HTTP 头信息
- ☐ C 路径参数（如 /items/{id}）
- ☐ D JSON 格式的请求体

答案

1=>D

FastAPI请求参数验证_Queryy方式



FastAPI 提供了强大的 Query 参数验证功能，主要通过 `Query` 类和 Pydantic 模型实现。以下是详细的验证方式分类和示例：

1. 基础验证

- 类型验证：自动将参数转换为声明类型（如 `int`、`str`）。

```
1 from fastapi import FastAPI, Query
2 app = FastAPI()
3
4 @app.get("/items/")
5 def read_items(q: str = Query(None)): # 默
   认可选
6     return {"q": q}
```

若传入非字符串类型（如 `/items/?q=123`），FastAPI 会自动处理为字符串；

若类型不匹配（如 `int` 参数传入非数字），返回 422 错误。

2. 长度限制

- 字符串长度：通过 `min_length` 和 `max_length` 限制。

```
1 @app.get("/items/")
2 def read_items(q: str = Query(None,
   min_length=3, max_length=50)):
3     return {"q": q}
```

若 `q` 长度不符合要求（如 `q=ab`），返回 422 错误。

3. 正则表达式验证

- 格式匹配：使用 `regex` 或者 `pattern` 参数。

```
1 @app.get("/items/")
2 def read_items(q: str = Query(None,
   regex="^fixedquery$")):
3     return {"q": q}
```

仅接受完全匹配 `fixedquery` 的输入。

4. 默认值与必填项

- 默认值：通过 `Query` 的第一个参数设置。

```
1 @app.get("/items/")
2 def read_items(q: str = Query("default")):
3     return {"q": q}
```

- 必填参数：使用 `...`（省略号）标记。

```
1 @app.get("/items/")
2 def read_items(q: str = Query(...,
   min_length=3)):
3     return {"q": q}
```

若未提供 `q`，返回 422 错误。

5. 数值范围验证

- 数值限制：通过 `gt`（大于）、`lt`（小于）等参数。

```
1 @app.get("/users/")
2 def get_users(age: int = Query(..., gt=0,
  lt=100)):
3     return {"age": age}
```

若 `age` 不在 0-100 之间，返回错误。

6. 多值参数（列表）

- 接收多个值：使用 `List` 类型。

```
1 from typing import List
2 @app.get("/items/")
3 def read_items(q: List[str] =
  Query(["default"])):
4     return {"q": q}
```

访问 `/items/?q=foo&q=bar` 时，`q` 值为 `["foo", "bar"]`。

7. 别名与元数据

- 别名：解决参数名冲突或提供友好名称。

```
1 @app.get("/items/")
2 def read_items(q: str = Query(None,
  alias="item-query")):
3     return {"q": q}
```

需通过 `/items/?item-query=foo` 访问。

- 描述信息：通过 `description` 参数添加文档说明。

```
1 @app.get("/items/")
2 def read_items(q: str = Query(None,
  description="搜索关键词")):
3     return {"q": q}
```

8. 弃用参数

- 标记弃用：通过 `deprecated=True`。

```
1 @app.get("/items/")
2 def read_items(q: str = Query(None,
3     deprecated=True)):
4     return {"q": q}
```

在文档中标记该参数已弃用。

实时效果反馈

1. 在 FastAPI 中，以下哪种正则表达式验证方式是正确的？

- ☒ A `regex="^fixedquery$"`
- ☐ B `pattern="^[A-Z]+$"`
- ☐ C 两者均可
- ☐ D 仅支持 `regex`

答案

1=>C

FastAPI请求参数验证_Path方式



FastAPI 中的 Path 参数验证主要通过 `Path` 类和 Pydantic 模型实现，以下是详细的分类和示例：

1. 基础类型验证

- 自动类型转换：FastAPI 根据类型注解自动转换路径参数类型，失败则返回 422 错误。

```
1 from fastapi import FastAPI
2 app = FastAPI()
3
4 @app.get("/items/{item_id}")
5 def read_item(item_id: int): # 自动验证为整数
6     return {"item_id": item_id}
```

访问 `/items/123` 返回 `{"item_id": 123}`，但 `/items/abc` 会触发 422 错误。

2. 必填与可选

- 强制必填：路径参数默认必填，即使设置 `None` 也无效。

```
1 @app.get("/items/{item_id}")
2 def read_item(item_id: int = Path(...)):
3     # 显式声明必填
4     return {"item_id": item_id}
```

未提供 item_id 时直接报错。

3. 数值范围验证

- 范围限制：通过 gt（大于）、lt（小于）等参数限制数值范围。

```
1 from fastapi import Path
2
3 @app.get("/products/{product_id}")
4 def get_product(product_id: int =
  Path(..., gt=1000, le=10000)):
5     return {"product_id": product_id}
```

仅接受 $1000 < \text{product_id} \leq 10000$ 的值。

4. 字符串格式验证

- 正则表达式：使用 regex 或 pattern 校验字符串格式。

```
1 @app.get("/credit-cards/{card_no}")
2 def get_card(card_no: str = Path(...,
  regex=r"^[4-6]\d{15}$")):
3     return {"card_no": card_no}
```

仅接受以 4-6 开头且长度为 16 的数字字符串。

5. 枚举值限制

- 预设值：通过 Enum 类限制参数为特定值。

```

1 from enum import Enum
2
3 class ModelName(str, Enum):
4     alexnet = "sxt"
5     resnet = "itbaizhan"
6
7 @app.get("/models/{model_name}")
8 def get_model(model_name: ModelName):
9     return {"model_name": model_name}

```

仅接受 sxt或 itbaizhan

5. 元数据与别名

- 描述与别名：添加文档说明或解决命名冲突。

```

1 @app.get("/users/{user_id}")
2 def read_user(user_id: int = Path(...,
3     title="用户ID", alias="user-id")):
4     return {"user_id": user_id}

```

访问时需使用 /users/123，但文档中显示别名为 user-id。

6. 自定义验证器

- 复杂逻辑

```

1 from typing import Annotated
2 from pydantic import BeforeValidator
3
4 def validate(value):
5     if not value.startswith('P-'):
6         raise ValueError('必须以P-开头')
7     return value
8
9

```

```
10 # 创建带验证的类型别名
11 Item = Annotated[str,
    BeforeValidator(validate)]
12
13
14 @app.get('/items6/{item_id}')
15 def read_item6(item_id: Item):
16     return {'item_id': item_id}
17
```

仅接受以 P-开头的编码。

实时效果反馈

1. 在 FastAPI 中，若需限制路径参数 `product_id` 必须大于 1000，应如何定义？

- ☒ A `product_id: int = Path(gt=1000)`
- ☐ B `product_id: int = Path(..., ge=1001)`
- ☐ C 以上均可

答案

1=>C

FastAPI请求参数验证_Field方式



FastAPI 中的 `Field` 是 Pydantic 提供的核心验证工具，用于为模型字段添加校验规则和元数据。以下是其详细使用方式分类和示例：

1. 基础类型与默认值

- **类型注解**：通过 Python 类型提示声明字段类型，自动触发类型验证。
- **默认值**：通过 `default` 参数设置字段默认值。

```
1 from pydantic import BaseModel, Field
2
3 class User(BaseModel):
4     name: str = Field(default="Anonymous")
5     # 默认值为 "Anonymous"
6     age: int = Field(...) # 必填字段
```

若 `age` 未提供，FastAPI 返回 422 错误。

2. 数值范围验证

- **范围限制**：使用 `gt`（大于）、`ge`（大于等于）、`lt`（小于）、`le`（小于等于）。

```
1 class Product(BaseModel):
2     price: float = Field(..., gt=0,
3                             le=1000) # 价格必须 >0 且 ≤1000
```

若 `price=0`，触发验证错误。

3. 字符串格式验证

- **长度限制**: `min_length` 和 `max_length`。
- **正则表达式**: `regex` 或 `pattern`。

```
1 class Account(BaseModel):  
2     username: str = Field(...,  
   min_length=3, max_length=20)  
3     password: str = Field(...,  
   regex=r"^[A-Za-z0-9@#$$%^&+=]{8,}$")
```

密码需至少 8 位且包含特定字符。

4. 元数据与文档增强

- **标题与描述**: `title` 和 `description` 用于生成 OpenAPI 文档。
- **示例值**: `example` 提供文档中的样例。

```
1 class Item(BaseModel):  
2     name: str = Field(..., title="商品名称",  
   description="必填, 长度不超过50字符",  
   example="手机")
```

文档中显示字段说明和示例。

5. 复杂类型与嵌套验证

- **列表验证**: `min_items`、`max_items` 限制列表长度。
- **嵌套模型**: 结合其他 Pydantic 模型。

```
1 class Order(BaseModel):  
2     items: list = Field(..., min_items=1)  
   # 至少1个商品  
3     address: dict = Field(...,  
   description="配送地址")
```

嵌套模型支持递归验证。

6. 自定义验证器

- 字段级验证：使用

```
1 @field_validator
```

装饰器。

```
1 from pydantic import field_validator
2
3 class User(BaseModel):
4     email: str
5
6     @field_validator("email")
7     def validate_email(cls, v):
8         if "@" not in v:
9             raise ValueError("邮箱格式无效")
10        return v
```

自定义逻辑优先于 Field 的内置规则。

7. 别名与字段映射

- 别名：alias 解决 JSON 键名与 Python 变量名不一致问题。

```
1 class Data(BaseModel):
2     user_name: str = Field(...,
3     alias="userName") # 接收 JSON 中的
4     "userName"
```

请求体需包含 {"userName": "John"}。

8. 高级约束

- 枚举值：结合 Enum类限制字段取值。

```
1 from enum import Enum
2
3 class Status(str, Enum):
4     ACTIVE = "active"
5     INACTIVE = "inactive"
6
7 class Task(BaseModel):
8     status: Status =
9     Field(default=Status.ACTIVE)
```

仅接受 active 或 inactive。

9. 动态默认值

- 默认工厂：default_factory 动态生成默认值。

```
1 from uuid import uuid4
2
3 class Document(BaseModel):
4     id: str =
5     Field(default_factory=lambda:
6           str(uuid4()))
```

每次创建模型时生成唯一 ID。

实时效果反馈

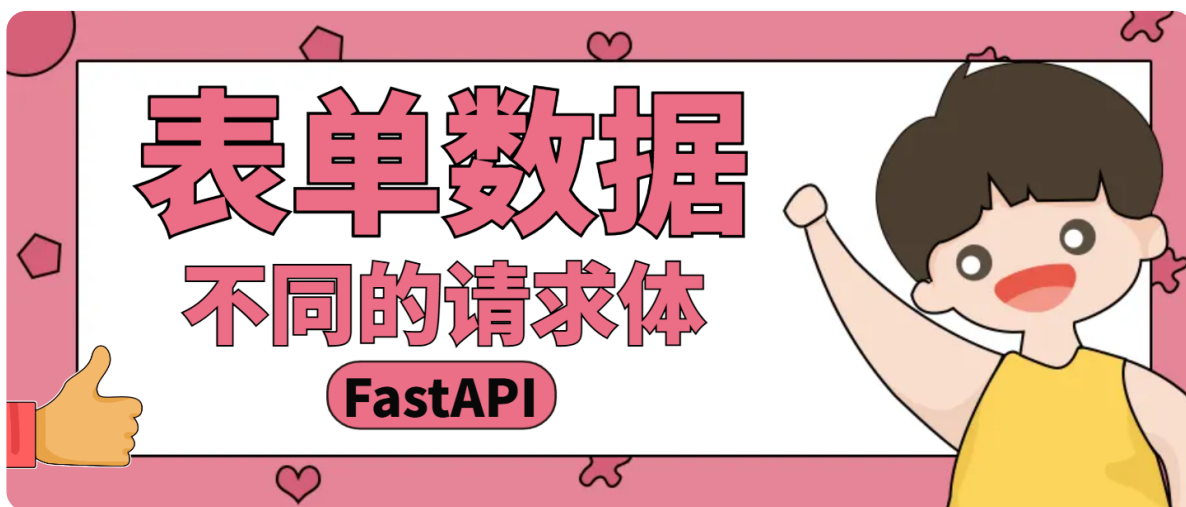
1. 在 FastAPI 中，如何限制字符串字段长度在 3 到 20 之间？

- ☒ A Field(min=3, max=20)
- ☐ B Field(min_length=3, max_length=20)
- ☐ C Field(length=(3, 20))
- ☐ D Field(regex="^{3,20}\$")

答案

1=>B

FastAPI表单数据



要使用表单，需预先安装python-multipart

```
1 pip install python-multipart==0.0.20
```

提示

自 FastAPI 版本 **0.113.0** 起支持此功能

```
1 # 表单数据
2 from typing import Annotated
3 from fastapi import FastAPI, Form
4 from pydantic import BaseModel
5
```

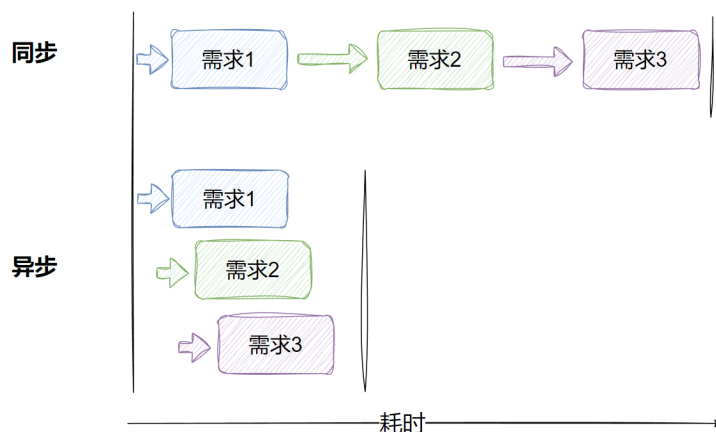
```
6 app = FastAPI()
7
8
9 @app.post("/login1")
10 def login1(username: str = Form(...),
11 password: str = Form(...)):
12     return {"username": username,
13 "password": password}
14
15 class User1(BaseModel):
16     username: str
17     password: str
18
19 @app.post("/login2")
20 def login2(user: Annotated[User1, Form()]):
21     return user
22
23
24 class User2(BaseModel):
25     username: str = Form(...)
26     password: str = Form(...)
27
28
29 @app.post("/login3")
30 def login3(user: Annotated[User2, Form()]):
31     return user
32
33
34 if __name__ == '__main__':
35     import uvicorn
```

```
uvicorn.run(app='main11:app',  
host='127.0.0.1', port=8000, reload=True)
```

FastAPI异步处理



在 FastAPI 中，异步（async）和非异步（同步）编程方式是其核心特性之一，它们在处理请求、性能以及并发能力上有着显著的区别。



异步 (async def)

- **事件循环**：异步代码运行在 Python 的异步事件循环（如 asyncio）中。事件循环负责协调多个协程（coroutines），在某个协程等待 I/O 操作（如数据库查询或 HTTP 请求）时，事件循环可以切换到其他协程执行。
- **非阻塞**：当一个异步函数调用 await，它会暂停执行，将控制权交回事件循环，允许其他任务运行，直到等待的操作完成。
- **并发性**：异步模式允许单个线程处理大量并发请求，特别适合高并发场景（如 Web 服务器处理大量客户端请求）。

非异步 (def)

- **阻塞式执行**：同步函数在调用时会完全占用线程，直到函数执行完成才会释放线程。
- **线程池**：在 FastAPI 中，同步函数由工作线程（worker threads）处理，Uvicorn（FastAPI 常用的 ASGI 服务器）会将同步函数放入线程池运行。
- **并发限制**：线程池的大小限制了同步函数的并发能力。如果线程池耗尽（例如，处理大量阻塞请求），新请求将排队等待。

示例

```
1 from fastapi import FastAPI
2 import asyncio
3 import time
4
5 app = FastAPI()
6
7 # 异步 endpoint: 模拟并发 I/O 操作
8 @app.get("/async")
9 async def async_endpoint():
10     start = time.time()
11     # 模拟 5 次异步 I/O 操作（并发执行）
12     tasks = [asyncio.sleep(1) for _ in range(5)]
13     await asyncio.gather(*tasks)
14     end = time.time()
15     return {"异步时长": f"{end - start:.2f}秒"}
16
17 # 同步 endpoint: 模拟相同的 I/O 操作
18 @app.get("/sync")
19 def sync_endpoint():
20     start = time.time()
21     # 模拟 5 次同步 I/O 操作（顺序执行）
22     for _ in range(5):
23         time.sleep(1)
```

```
24     end = time.time()
25     return {"同步时长": f"{end - start:.2f}
    秒"}
26
27
28 if __name__ == "__main__":
29     import uvicorn
30     uvicorn.run('app:app', host="127.0.0.1",
    port=8000, reload=True)
31
```

FastAPI文件上传



1. 基础文件上传

方式一: `bytes` 类型 (适合小文件)

```

1 from fastapi import FastAPI, File
2
3 app = FastAPI()
4
5 @app.post("/upload/")
6 def upload_file(file: bytes = File(...)):
7     return {"file_size": len(file)} # 文件内
    容直接加载到内存

```

特点:

- 文件以二进制形式读取，适合小于 10MB 的文件
- 内存占用高，大文件可能导致崩溃

方式二: `UploadFile` 类型 (推荐)

```

1 from fastapi import UploadFile
2
3 @app.post("/upload/")
4 def upload_file(file: UploadFile):
5     contents = await file.read() # 异步读取
6     return {"filename": file.filename,
    "size": len(contents)}

```

优势:

- 自动处理内存和磁盘存储 (超过阈值存磁盘)
- 支持文件元数据 (`filename` , `content_type`)
- 提供异步文件操作方法 (`read()` , `write()`)

2. 多文件上传

```
1 from typing import List
2
3 @app.post("/batch-upload/")
4 def batch_upload(files: List[UploadFile] =
  File(...)):
5     return {"count": len(files), "names":
      [f.filename for f in files]}
```

注意:

- 前端需设置 `<input type="file" multiple>`
- 每个文件独立处理，避免内存溢出

3. 文件保存到本地

同步保存（小文件）

```
1 @app.post("/save-file/")
2 async def save_file(file: UploadFile):
3     with open(file.filename, "wb") as f:
4         f.write(await file.read()) # 同步写入
5     return {"status": "saved"}
```

异步保存（大文件优化）

```
1 import aiofiles
2
3 @app.post("/save-large-file/")
4 async def save_large_file(file: UploadFile):
5     async with aiofiles.open(file.filename,
6                               "wb") as f:
7         while chunk := await file.read(1024 *
8                                         1024): # 分块读取1MB
9             await f.write(chunk)
10    return {"status": "success"}
```

最佳实践:

- 使用 `aiofiles` 异步 IO 提升性能
- 分块读取避免内存溢出

4. 文件验证

类型校验

```
1 from fastapi import HTTPException
2
3 ALLOWED_EXTENSIONS = {".jpg", ".jpeg",
4                        ".gif"}
5
6 @app.post("/upload-image/")
7 def upload_image(file: UploadFile):
8     # 校验文件类型
9     ext = Path(file.filename).suffix.lower()
10    if ext not in ALLOWED_EXTENSIONS:
11        raise HTTPException(400, "不支持的文件
12                                扩展名")
13
14    return {"filename": file.filename}
```

5. 混合表单与文件上传

```
1 from fastapi import Form
2
3 @app.post("/submit-form/")
4 def submit_form(
5     username: str = Form(...),
6     avatar: UploadFile = File(...)
7 ):
8     return {"user": username, "avatar_size":
9         avatar.size}
```

关键点:

- 必须使用 `multipart/form-data` 编码
- 不能与 JSON 请求体 (`Body`) 混用

实时效果反馈

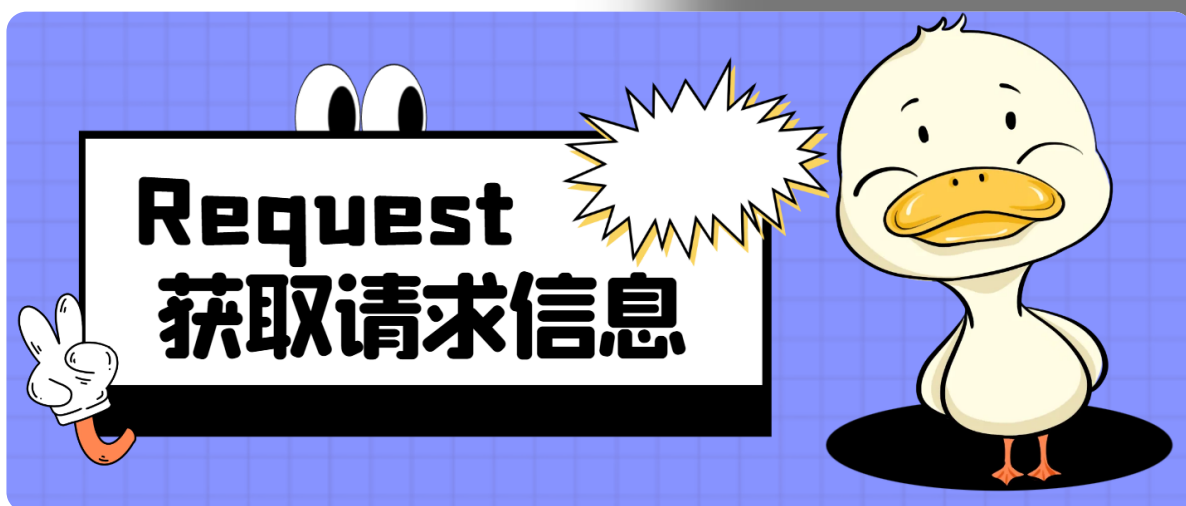
1. 大文件上传时，以下哪项操作可能导致服务器崩溃？

- ☒ A 使用 `UploadFile` 分块读取
- ☐ B 用 `bytes` 直接加载10GB文件到内存
- ☐ C 限制文件大小为5MB
- ☐ D 异步保存到磁盘

答案

1=>B

FastAPI请求对象Request



基础用法

通过注入 `Request` 对象可获取完整的请求信息景：

```
1  # Request获取请求信息
2
3  from fastapi import FastAPI, Request
4
5  app = FastAPI()
6
7
8  @app.get('/client-info')
9  async def client_info(request: Request):
10     return {
11         "请求URL": request.url,
12         "请求方法": request.method,
13         "请求IP": request.client.host,
14         "请求参数": request.query_params,
15         "请求头": request.headers,
16         # "请求json": await request.json(),
```

```
17         "请求cookies": request.cookies,  
18         # "请求form": await request.form(),  
19         # "请求files": request.files,  
20         "请求path_params":  
request.path_params,  
21     }  
22  
23  
24  
25 if __name__ == '__main__':  
26     import uvicorn  
27     uvicorn.run(app='main14:app',  
host='127.0.0.1', port=8000, reload=True)
```

FastAPI响应数据-JSON格式



响应类型介绍

在企业级应用中，以下响应类型最为常见：

- 1 **JSON 响应**：用于 RESTful API 的数据交互，占主导地位（如用户信息、订单、配置）。
- 2 **列表响应**：用于分页查询或批量数据返回（如商品列表、日志记录）。
- 3 **文件响应**：用于报表导出、文件下载（如 CSV、PDF）。

- 4 **字符串响应**：用于健康检查或简单状态反馈。
- 5 **HTML 响应**：用于管理后台或简单的 Web 页面。
- 6 **重定向响应**：用于认证流程或 URL 迁移。
- 7 **流式响应**：用于实时数据传输或大文件处理。

JSON 响应 (80%+ 场景)

JSON 是 Web API 中最常见的响应格式，FastAPI 天然支持通过返回 Python 字典或 Pydantic 模型自动序列化为 JSON 响应

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3
4 app = FastAPI()
5
6 class Item(BaseModel):
7     id: int
8     name: str
9     tags: list[str] = []
10
11 # 直接返回字典（自动序列化）
12 @app.get("/items/dict")
13 async def get_item_dict():
14     return {"id": 1, "name": "Foo"}
15
16 # 使用 Pydantic 模型（推荐企业实践）
17 @app.get("/items/model",
18         response_model=Item)
19 async def get_item_model():
20     return Item(id=2, name="Bar")
```

企业级技巧:

- 通过 `response_model` 实现响应数据校验和 OpenAPI 文档自动生成

- 使用 `response_model_exclude_unset=True` 过滤未设置的默认值字段
- 组合模型: `response_model=Union[SuccessResponse[Item], ErrorResponse]`

实时效果反馈

1. Web API 中最常见的响应格式是什么?

- ☒ A HTML 响应
- ☐ B JSON 响应
- ☐ C 文件响应
- ☐ D 流式响应

2. 在 FastAPI 中, 哪个功能被推荐用于企业级应用以确保响应数据验证和自动生成 OpenAPI 文档?

- ☒ A `response_model`
- ☐ B `response_model_exclude_unset`
- ☐ C `Union[SuccessResponse, ErrorResponse]`
- ☐ D `Pydantic BaseModel`

答案

1=>B 2=>A



列表响应是指 API 返回一组数据的响应，通常以 JSON 格式返回一个数组或包含数组的对象。常见场景包括：

- **分页查询**：返回数据的一部分（如每页 10 条记录），避免一次性加载所有数据。
- **批量数据返回**：返回符合条件的全部或部分数据（如所有订单、日志）。
- 列表响应通常包含：
 - **数据列表**：核心数据（例如商品列表、日志记录）。
 - **分页元数据**：如总记录数、当前页码、每页记录数、总页数等。
 - **状态信息**：如请求状态（成功或失败）。
 - **过滤/排序信息**：描述当前返回的数据是如何过滤或排序的

示例

```

1 from fastapi import FastAPI, Query
2 from pydantic import BaseModel
3 from typing import List, Optional
4 from fastapi.responses import JSONResponse
5
6 app = FastAPI()
7
8 # 定义商品模型
9 class Item(BaseModel):
10     id: int

```

```
11     name: str
12     price: float
13     category: str
14 # 定义分页元数据模型
15 class Pagination(BaseModel):
16     total: int
17     page: int
18     page_size: int
19     total_pages: int
20 # 定义列表响应模型
21 class ListResponse(BaseModel):
22     status: str = "success"
23     data: List[Item]
24     pagination: Pagination
25 # 模拟数据库数据
26 DB = [
27     Item(id=i, name=f"Apple {i}",
28 price=100.0 * i, category="ipad" if i % 2 ==
29 0 else "iphone")
30     for i in range(1, 101)
31 ]
32 # 分页查询商品列表
33 @app.get("/items1")
34 async def get_items1():
35     return ['Apple1', 'Apple2', 'Apple3']
36 @app.get("/items2")
37 async def get_items2():
38     return DB
39 @app.get("/items3",
40 response_model=ListResponse)
41 async def get_items3(
```

```
40     page: int = Query(1, ge=1,
description="页码"),
41     page_size: int = Query(
42         10, ge=1, le=100, description="数
量"),
43     category: Optional[str] = Query(None,
description="分类")
44 ):
45     # 过滤数据
46     filtered_items = DB
47     if category:
48         filtered_items = [
49             item for item in DB if
item.category == category]
50     # 计算分页参数
51     total = len(filtered_items)
52     total_pages = (total + page_size - 1) //
page_size
53     start = (page - 1) * page_size
54     end = start + page_size
55     # 确保页面有效
56     if start >= total and total > 0:
57         return JsonResponse(
58             status_code=400,
59             content={"status": "error",
"message": "错误的页码"}
60         )
61     # 获取分页数据
62     paginated_items =
filtered_items[start:end]
63     # 返回响应
64     return ListResponse(
```

```
65         data=paginated_items,  
66         pagination=Pagination(  
67             total=total,  
68             page=page,  
69             page_size=page_size,  
70             total_pages=total_pages  
71         )  
72     )
```

实时效果反馈

1. 列表响应的主要目的是什么？

- A** 返回单个数据记录的详细信息
- B** 返回一组数据的集合，通常以数组形式
- C** 更新服务器上的数据
- D** 删除数据库中的记录

答案

1=>B

FastAPI响应数据-文件格式



文件响应是指 API 在响应客户端请求时，返回一个文件内容的 HTTP 响应，通常包含文件的二进制数据或文本数据。

文件响应的关键特点：

- **内容类型**：通过 Content-Type 头指定文件的 MIME 类型，如 application/pdf (PDF 文件)、text/csv (CSV 文件)、application/vnd.ms-excel (Excel 文件)。
- **文件内容**：响应的 body 包含文件的实际数据，可能是二进制（如 PDF、图片）或文本（如 CSV、JSON 文件）。

示例

```
1 # 文件响应
2 from fastapi import Response
3 from fastapi.responses import
  StreamingResponse
4 from fastapi.responses import FileResponse
5 from fastapi import FastAPI
6
7 app = FastAPI()
8
9
10 @app.get("/download")
11 async def download_file():
12     return FileResponse(
13         path="data/fastapi 文档.pdf",
```

```
14         filename="file.pdf", # 客户端看到的文
      件名
15         media_type="application/pdf" # 显式
      设置 MIME 类型
16         # path="data/AI变声器.mp4",
17         # filename="file2.mp4", # 客户端看到
      的文件名
18         # media_type="video/mp4" # 显式设置
      MIME 类型
19     )
20
21
22 def generate_chunks(file_path: str,
23 chunk_size: int = 1024 * 1024 * 10):
24     with open(file_path, "rb") as f:
25         while chunk := f.read(chunk_size):
26             yield chunk
27
28 @app.get("/stream-video")
29 async def stream_video():
30     return StreamingResponse(
31         content=generate_chunks("data/AI变声
      器.mp4"),
32         media_type="video/mp4",
33     )
34
35
36 @app.get("/custom-file")
37 async def custom_file():
38     binary_data = b"File content"
39     return Response(
```

```
40         content=binary_data,  
41         media_type="text/plain",  
42         headers={"Content-Disposition":  
            "attachment; filename=log.txt"}  
43     )  
44  
45 if __name__ == '__main__':  
46     import uvicorn  
47     uvicorn.run('main17:app',  
        host="127.0.0.1", port=8000, reload=True)  
48
```

FastAPI响应数据-其它格式



其它响应的格式

- 1 字符串响应
- 2 重定向
- 3 HTML 响应
- 4 静态文件

```
1 # 其响应
2 from fastapi import FastAPI
3 from fastapi.responses import
  RedirectResponse, HTMLResponse
4 from fastapi.staticfiles import StaticFiles
5
6 app = FastAPI()
7
8 @app.get("/hello")
9 async def hello():
10     return "Hello, world!" # 默认 Content-
  Type: text/plain; charset=utf-8
11
12 @app.get("/redirect")
13 async def redirect1():
14     return RedirectResponse("/hello")
15
16 @app.get("/items")
17 async def create_item(name: str): # 查询参数
18     return {"name": name}
19
20 @app.get("/redirect2")
21 async def redirect2():
22     return RedirectResponse("/items?name=张
  三")
23
24 @app.get("/success",
  response_class=HTMLResponse)
25 async def success():
26     return "<html><body><h1>操作成功! </h1>
  </body></html>"
27
```

```
28 # 挂载静态目录到 /static 路径
29 app.mount("/static",
    StaticFiles(directory="static"),
    name="static")
30
31 app.mount("/app",
    StaticFiles(directory="template",
    html=True),
    name="页面"
32 )
33
34
```

FastAPI 的 ORM 生态



ORM介绍

ORM (Object-Relational Mapping, 对象关系映射) 是一种编程技术, 用于在面向对象编程语言和关系型数据库之间建立映射。它允许开发者通过操作对象的方式来与数据库进行交互, 而无需直接编写复杂的 SQL 语句。主要特点包括:

- 1 **对象与数据库表的映射:** ORM 将数据库中的表映射为编程语言中的类, 每一行数据对应一个对象, 表的列对应对象的属性

- 2 **简化数据库操作**：开发者可以使用面向对象的方法（如创建、查询、更新、删除对象）来操作数据库，而无需手动编写 SQL
- 3 **跨数据库兼容**：ORM 通常支持多种数据库（如 MySQL、PostgreSQL、SQLite），通过统一的接口减少数据库切换的成本
- 4 **提高开发效率**：通过自动化 SQL 生成和查询优化，减少重复代码，提升开发速度

工作原理

- ORM 框架定义了一个映射层，将类和数据库表关联起来
- 开发者通过 ORM 的 API 操作对象，ORM 自动将操作翻译成 SQL 语句，执行数据库操作并返回结果

优点

- 提高代码可读性和维护性
- 减少直接 SQL 操作带来的错误。
- 支持复杂查询和关系（如一对多、多对多）

缺点

- 性能可能略低于原生 SQL（因抽象层开销）
- 对于非常复杂的查询，可能需要直接编写 SQL

ORM工具介绍

- **SQLAlchemy**（同步/异步）：**≈80% 企业项目首选**，功能完备、社区成熟，支持复杂查询和事务管理。
- **Tortoise**（异步）：语法类似 Django ORM，适合异步优先项目，集成简便
- **GINO**（异步）：轻量级，基于 SQLAlchemy Core 的异步扩展，适合高性能 API

选型建议

- 传统企业项目 → **SQLAlchemy**（成熟稳定）
- 全异步微服务 → **Tortoise-ORM** 或 **GINO**

提示

FastAPI 无官方 ORM

实时效果反馈

1. 什么是 ORM 的主要功能？

- A** 直接编写 SQL 语句来操作数据库

- B** 将数据库表映射为编程语言中的类，实现对象操作数据库
- C** 优化数据库的存储结构
- D** 替换数据库管理系统

2. 以下哪项不是 ORM 的优点？

- A** 提高代码可读性和维护性
- B** 减少直接 SQL 操作带来的错误
- C** 性能总是优于原生 SQL
- D** 支持复杂查询和关系

答案

1=>B 2=>C

Tortoise-ORM配置



为什么选择 Tortoise-ORM

- 异步支持，与 FastAPI 无缝集成
- 简单易用的模型定义，类似 Django ORM
- 支持复杂关系（一对一、一对多、多对多）
- 自动生成表结构，适合快速开发

环境配置

安装必要的依赖：

```
1 pip install tortoise-orm==0.25.0
   aerich==0.9.0 aiomysql==0.2.0 tomlkit==0.13.2
```

配置数据库连接

Tortoise-ORM 需要一个配置字典来定义数据库连接信息，通常存储在 settings.py 或类似的配置文件中。

```
1 from typing import Dict
2
3 # Tortoise-ORM 配置
4 TORTOISE_ORM: Dict = {
5     "connections": {
6         # 开发环境使用 SQLite（基于文件，无需服务器）
7         "default": "sqlite://db.sqlite3",
8         # 生产环境示例：PostgreSQL
9         # "default":
10        "postgres://user:password@localhost:5432/dbname",
11        # 生产环境示例：MySQL
12        # "default":
13        "mysql://user:password@localhost:3306/dbname",
14    },
15 }
```

```

13     "apps": {
14         "models": {
15             "models": ["app.models",
16 "aerich.models"], # 模型模块和 Aerich 迁移模型
17             "default_connection": "default",
18         },
19         # 连接池配置（推荐）
20         "use_tz": False, # 是否使用时区
21         "timezone": "UTC", # 默认时区
22         "db_pool": {
23             "max_size": 10, # 最大连接数
24             "min_size": 1, # 最小连接数
25             "idle_timeout": 30 # 空闲连接超时
26             (秒)
27         }
28     }

```

配置说明

- **connections**: 定义数据库连接字符串
 - SQLite: `sqlite://db.sqlite3` (开发环境, 文件存储在项目根目录)
 - PostgreSQL: `postgres://user:password@host:port/dbname` (生产环境)
 - MySQL: `mysql://user:password@host:port/dbname` (生产环境)
- **apps**: 定义应用模块, models 列表包含模型文件路径和 Aerich 的迁移模型
- **db_pool**: 连接池参数, 优化数据库连接管理, 适合高并发场景
- **use_tz** 和 **timezone**: 控制时间字段的时区行为 (生产环境建议启用)

初始化 FastAPI 应用

```

1 from fastapi import FastAPI

```

```
2 from tortoise.contrib.fastapi import
   register_tortoise
3 from app.core.config import TORTOISE_ORM
4
5 app = FastAPI(title="FastAPI with Tortoise-
   ORM")
6
7 # 注册 Tortoise-ORM
8 register_tortoise(
9     app,
10    config=TORTOISE_ORM,
11    generate_schemas=True, # 开发环境自动生成
   表结构
12    add_exception_handlers=True, # 添加默认异
   常处理器
13 )
14
15 @app.get("/")
16 async def root():
17     return {"message": "Welcome to FastAPI
   with Tortoise-ORM!"}
```

实时效果反馈

1. 为什么选择 Tortoise-ORM 的一个主要原因是？

- ☒ A 它只支持同步操作
- ☐ B 它与 FastAPI 无缝集成并支持异步操作
- ☐ C 它不支持复杂关系
- ☐ D 它需要手动生成表结构

2. 在配置 Tortoise-ORM 时，以下哪项是正确的数据库连接字符串示例？

- A mysql://user:password@localhost:3306/dbname
- B <http://localhost:5432/dbname>
- C sqlite://user:password@db.sqlite3
- D postgres://dbname@localhost:5432

答案

1=>B 2=>A

Aerich 迁移工具的使用



Aerich 是 Tortoise-ORM 的数据库迁移工具，用于管理数据库结构的变更

Aerich 初始化

在项目根目录运行以下命令：

```
1 aerich init -t main.TORTOISE_ORM
```

这将生成：

- **pyproject.toml**: Aerich 配置文件，指定迁移配置。
- **migrations/**: 迁移文件目录，存放生成的 .sql 文件。

```
1 aerich init-db
```

生成和应用迁移

- ① 生成迁移文件：当模型发生变更时，运行以下命令生成迁移文件：

```
1 aerich migrate --name "info"
```

- ② 应用迁移：运行以下命令将迁移应用到数据库：

```
1 aerich upgrade
```

- ③ 验证迁移：检查迁移历史

```
1 aerich history
```

- ④ 回滚迁移：回退到指定版本

```
1 aerich downgrade
```

模型案例

```
1 from tortoise.models import Model
2 from tortoise.fields import CharField,
  DatetimeField, BooleanField
3
4 class User(Model):
```

```
5     id = CharField(max_length=36, pk=True)
    # 主键, UUID 字符串
6     username = CharField(max_length=50,
unique=True) # 用户名, 唯一
7     email = CharField(max_length=255,
unique=True) # 邮箱, 唯一
8     is_active = BooleanField(default=True)
    # 是否激活
9     created_at =
DatetimeField(auto_now_add=True) # 创建时间
10    updated_at =
DatetimeField(auto_now=True) # 更新时间
11
12    class Meta:
13        table = "users" # 自定义表名
14        ordering = ["-created_at"] # 默认按创
    建时间降序排序
15
16    def __str__(self):
17        return self.username
```

实时效果反馈

1. Aerich 的主要功能是什么?

- A** 提供异步 ORM 的模型定义
- B** 自动生成 FastAPI 路由
- C** 优化数据库连接池
- D** 管理数据库结构的变更

答案

1=>D

Tortoise-ORM 的模型定义



模型定义基础

Tortoise-ORM 的模型通过继承 `tortoise.models.Model` 类定义，每个模型对应数据库中的一张表。

字段定义使用 Tortoise-ORM 提供的字段类型，结合字段参数来指定约束和行为。

基本模型示例

以下是一个简单的 User 模型示例，展示模型定义的基本结构：

```
1 from tortoise.models import Model
2 from tortoise import fields
3
4 class User(Model):
```

```

5     id = fields.IntegerField(pk=True) # 主关键字
      段, 自动递增
6     username =
fields.CharField(max_length=50, unique=True)
      # 唯一用户名
7     email = fields.CharField(max_length=100,
index=True) # 带索引的邮箱
8     created_at =
fields.DateTimeField(auto_now_add=True) #
      自动记录创建时间
9     is_active =
fields.BooleanField(default=True) # 账户激活
      状态
10    credit =
fields.DecimalField(max_digits=10,
decimal_places=2, default=0.0) # 精确数值
11
12    class Meta:
13        table = "auth_users" # 自定义表名
14        ordering = ["-created_at"] # 默认按创
      建时间倒序

```

说明

- **继承 Model**: 所有模型必须继承 tortoise.models.Model。
- **字段定义**: 每个字段对应数据库表中的一列, 使用 Tortoise-ORM 的字段类 (如 CharField、BooleanField)。
- **Meta 类**: 用于定义模型的元数据, 如表名、排序规则等。

核心字段类型详解

Tortoise-ORM 提供了多种字段类型, 用于定义数据库列的类型和行为。以下是常用的字段类型及其典型用途:

Python 字段类型	数据库类型	描述	常用参数
CharField	VARCHAR	字符串	max_length (必填)、unique=True、index=True
TextField	TEXT	长文本	null=True (允许为空)
IntegerField / BigIntegerField	INTEGER/BIGINT	整数	pk=True (主键)、default=0
FloatField	DOUBLE/REAL	浮点数	gt=0 (值需大于0)
DecimalField	DECIMAL	高精度小数	max_digits=10, decimal_places=2 (最大10位, 含2位小数)
BooleanField	BOOLEAN	布尔值	default=True
DateField	DATE	日期	auto_now=True (保存时更新)
DatetimeField	DATETIME/TIMESTAMP	日期时间	auto_now_add=True (仅首次保存记录时间)
JSONField	JSON/JSONB	存储字典或列表	encoder (自定义编码器)、decoder (自定义解码器)
UUIDField	UUID/CHAR(36)	唯一标识符	若为主键默认生成 UUID4
BinaryField	BLOB	二进制数据	不支持过滤或更新操作

字段参数详解

字段支持多种参数，用于定义约束、默认值和其他行为。以下是常用参数：

- max_length: 字符串字段的最大长度 (CharField 必填)
- null: 是否允许字段为空 (null=True 表示数据库允许 NULL)
- default: 字段默认值 (如 default=0、default=True)
- unique: 是否唯一 (unique=True 确保字段值在表中唯一)
- index: 是否创建索引 (index=True 提高查询性能)
- description: 字段描述 (用于文档或数据库注释)
- pk: 是否为主键 (pk=True 表示该字段是主键)
- validators: 自定义验证函数

```

1 from tortoise.validators import validator
2
3 def validate_credit(value):
4     if value < 0:
5         raise ValueError("信用值不能为负数")
6
7 class User(Model):
8     credit = fields.IntField(validators=
    [validate_credit]) # 自定义验证函数

```

模型元数据 (Meta 类)

Meta 类用于定义模型的元数据，控制表结构和查询行为。常用属性包括：

- table: 自定义数据库表名 (如 table="users")
- unique_together: 定义联合唯一约束 (如 unique_together=[("field1", "field2")])
- indexes: 定义索引 (如 indexes=[("field1", "field2")])
- ordering: 默认排序规则 (如 ordering=["-created_at", "username"])

```

1 class Event(Model):
2     name = CharField(max_length=100)
3     location = CharField(max_length=200)
4     date = DateField()
5
6     class Meta:
7         table = "events"
8         unique_together = [("name", "date")]
9         # 名称和日期联合唯一
10        indexes = [("location",)] # 为
        location 字段创建索引
        ordering = ["date"] # 按日期升序排序

```

实时效果反馈

1. 在 Tortoise-ORM 中，定义一个模型时必须继承哪个类？

- ☒ A tortoise.Model
- ☐ B tortoise.models.Model
- ☐ C tortoise.BaseModel
- ☐ D tortoise.orm.Model

2. 以下哪个字段类型适合存储高精度小数 (如货币金额) ？

A DecimalField

B FloatField

C IntField

D CharField

答案

1=>B 2=>B

Tortoise-ORM对数据的CRUD



以下是针对Student表的CRUD操作模块，包含详细的增删改查函数：

```
1 from tortoise import fields, models
2
3 class Student(models.Model):
4     """
5     学生模型，包含基本信息。
6     用于演示单表的增删改查操作。
```

```

7         """
8         id = fields.IntegerField(pk=True,
description="学生ID, 主键")
9         name = fields.CharField(max_length=50,
description="学生姓名")
10        age = fields.IntegerField(null=True,
description="学生年龄, 可为空")
11        email = fields.CharField(max_length=100,
unique=True, null=True, description="学生邮
箱, 唯一")
12
13        class Meta:
14            table = "students"
15
16        def __str__(self):
17            return f"Student: {self.name}, Age:
{self.age}, Email: {self.email}"

```

CRUD函数说明:

① 创建 (Create) : create_student

- 创建学生记录, 支持可选字段 (age和email)
- 检查email唯一性, 抛出IntegrityError如果违反约束

② 查询 (Read) :

- get_student: 根据id精确查询
- get_students_by_name: 支持模糊查询 (name__contains)
- get_all_students: 查询所有学生

③ 更新 (Update) : update_student

- 允许更新部分字段, 保持其他字段不变
- 支持将age或email设为null

④ 删除 (Delete) : delete_student

- 根据id删除学生记录
- 确保记录存在, 避免无效操作

```
1 from tortoise import Tortoise
2 from tortoise.exceptions import
  DoesNotExist, IntegrityError
3 from models import Student
4 import logging
5
6 # 配置日志
7 logging.basicConfig(level=logging.INFO)
8 logger = logging.getLogger(__name__)
9
10 async def create_student(name: str, age:
  int = None, email: str = None) -> Student:
11     """
12     创建学生记录
13     Args:
14         name: 学生姓名, 必填
15         age: 年龄, 可选
16         email: 邮箱, 可选但必须唯一
17     Returns:
18         创建的学生对象
19     """
20     try:
21         student = await
  Student.create(name=name, age=age,
  email=email)
22         logger.info(f"创建学生:
  {student.name}, 年龄: {student.age}, 邮箱:
  {student.email}")
23         return student
24     except IntegrityError as e:
25         logger.error(f"创建学生失败: {e} (可能
  是邮箱重复)")
```

```

26         raise
27     except Exception as e:
28         logger.error(f"创建学生时发生未知错误：
    {e}")
29         raise
30
31 async def update_student(student_id: int,
    name: str = None, age: int = None, email:
    str = None) -> Student:
32     """
33     更新学生信息
34     Args:
35         student_id: 学生ID
36         name: 新姓名, 可选
37         age: 新年龄, 可选
38         email: 新邮箱, 可选
39     Returns:
40         更新后的学生对象
41     """
42     try:
43         student = await
Student.get(id=student_id)
44         if name:
45             student.name = name
46         if age is not None: # 允许年龄设为
null
47             student.age = age
48         if email is not None: # 允许邮箱设为
null
49             student.email = email
50         await student.save()

```

```
51         logger.info(f"更新学生：
{student.name}, 年龄： {student.age}, 邮箱：
{student.email}")
52         return student
53     except DoesNotExist:
54         logger.error(f"学生ID {student_id}
不存在")
55         raise
56     except IntegrityError as e:
57         logger.error(f"更新学生失败： {e} (可能
是邮箱重复)")
58         raise
59     except Exception as e:
60         logger.error(f"更新学生时发生未知错误：
{e}")
61         raise
62
63 async def delete_student(student_id: int) -
> None:
64     """
65     删除学生记录
66     Args:
67         student_id: 学生ID
68     """
69     try:
70         student = await
Student.get(id=student_id)
71         await student.delete()
72         logger.info(f"删除学生：
{student.name}, ID: {student.id}")
73     except DoesNotExist:
```

```
74         logger.error(f"学生ID {student_id}  
不存在")  
75         raise  
76     except Exception as e:  
77         logger.error(f"删除学生时发生未知错误:  
{e}")  
78         raise  
79     async def get_student(student_id: int) ->  
Student:  
80         """  
81         根据ID查询学生  
82         Args:  
83             student_id: 学生ID  
84         Returns:  
85             学生对象  
86         """  
87         try:  
88             student = await  
Student.get(id=student_id)  
89             logger.info(f"查询学生:  
{student.name}, ID: {student.id}")  
90             return student  
91         except DoesNotExist:  
92             logger.error(f"学生ID {student_id}  
不存在")  
93             raise  
94         except Exception as e:  
95             logger.error(f"查询学生时发生未知错误:  
{e}")  
96             raise  
97
```

```

98  async def get_students_by_name(name: str) -
    > list[Student]:
99      """
100     根据姓名模糊查询学生
101     Args:
102         name: 学生姓名（支持部分匹配）
103     Returns:
104         匹配的学生列表
105     """
106     try:
107         students = await
Student.filter(name__contains=name)
108         logger.info(f"查询到 {len(students)}
个姓名包含 '{name}' 的学生")
109         return students
110     except Exception as e:
111         logger.error(f"模糊查询学生时发生错误：
{e}")
112         raise
113
114  async def get_all_students() ->
list[Student]:
115      """
116     查询所有学生
117     Returns:
118         所有学生列表
119     """
120     try:
121         students = await Student.all()
122         logger.info(f"查询到 {len(students)}
个学生")
123         return students

```

```
124         except Exception as e:
125             logger.error(f"查询所有学生时发生错误:
           {e}")
126             raise
```

Tortoise-ORM 的关联关系建立



需求

我们将模拟一个学校管理系统，包含以下实体：

- **学生 (Student)**：每个学生有唯一的个人信息档案（1对1）。
- **成绩 (Grade)**：一个学生可以有多份成绩记录（1对多）。
- **课程 (Course)**：学生和课程之间是多对多关系（通过成绩表关联）。

示例

```
1 from tortoise import fields, models
2
3 class Student(models.Model):
4     id = fields.IntField(pk=True)
5     name = fields.CharField(max_length=50)
```

```
6      # 1对1关系：学生和学生档案
7      profile = fields.OneToOneField(
8          "models.StudentProfile",
on_delete=fields.CASCADE,
related_name="student"
9      )
10     # 1对多关系：学生和成绩（通过反向关系访问）
11     grades = fields.ReverseRelation["Grade"]
12
13     class Meta:
14         table = "students"
15
16     def __str__(self):
17         return f"Student: {self.name}"
18
19 class StudentProfile(models.Model):
20     id = fields.IntegerField(pk=True)
21     student = fields.OneToOneField(
22         "models.Student",
on_delete=fields.CASCADE,
related_name="profile"
23     )
24     address =
fields.CharField(max_length=100, null=True)
25     phone = fields.CharField(max_length=20,
null=True)
26
27     class Meta:
28         table = "student_profiles"
29
30     def __str__(self):
```

```
31         return f"Profile for
{self.student.name}: {self.address},
{self.phone}"
32
33 class Course(models.Model):
34     id = fields.IntegerField(pk=True)
35     name = fields.CharField(max_length=50)
36     # 多对多关系: 通过Grade表关联学生
37     students = fields.ManyToManyField(
38         "models.Student", through="grades",
related_name="courses"
39     )
40
41     class Meta:
42         table = "courses"
43
44     def __str__(self):
45         return f"Course: {self.name}"
46
47 class Grade(models.Model):
48     id = fields.IntegerField(pk=True)
49     student = fields.ForeignKeyField(
50         "models.Student",
related_name="grades",
on_delete=fields.CASCADE
51     )
52     course = fields.ForeignKeyField(
53         "models.Course",
related_name="grades",
on_delete=fields.CASCADE
54     )
55     score = fields.FloatField()
```

```

56
57     class Meta:
58         table = "grades"
59         unique_together = ("student",
60                             "course") # 确保学生和课程组合唯一
61
62     def __str__(self):
63         return f"{self.student.name} - {self.course.name}: {self.score}"

```

1对1关系:

- Student.profile使用OneToOneField, 指向StudentProfile。
- on_delete=fields.CASCADE表示如果学生被删除, 其档案也会被删除。
- related_name="student"允许从StudentProfile反向访问Student。

1对多关系:

- Grade.student使用ForeignKeyField, 指向Student。
- Student.grades是反向关系, 自动生成, 允许访问学生的所有成绩。

多对多关系:

- Course.students使用ManyToManyField, 通过Grade表关联。
- through="grades"指定中间表, Grade表包含student和course外键。
- unique_together确保一个学生对一门课程只有一条成绩记录。

实时效果反馈

1. 在一对一关系中, 两个表之间的关联特点是什么?

- A** 一个表中的一条记录可以对应另一个表中的多条记录
- B** 一个表中的一条记录只对应另一个表中的一条记录
- C** 两个表中的记录可以任意组合, 无限制
- D** 一个表中的多条记录对应另一个表中的一条记录

2. 以下哪种场景适合使用多对多关系?

- A** 一个用户只能有一个邮箱地址，一个邮箱地址只能属于一个用户
- B** 一个部门可以有多个员工，一个员工只能属于一个部门
- C** 一个学生可以选修多门课程，一门课程可以被多个学生选修
- D** 一个订单只能属于一个客户，一个客户可以有多个订单

答案

1=>B 2=>C

Tortoise-ORM 的关联关系数据操作

关联关系结构 数据操作CRUD

Tortoise ORM



示例

```

1  from model24 import Student, Grade, Course
2  from tortoise import Tortoise, run_async
3
4
5  async def init():
6      await Tortoise.init(
7
8          db_url="mysql://root:123456@192.168.31.152:
9          3306/fastapi_db4",
10         modules={"models": ["model24"]}
11     )
12
13  async def create_data():
14      await init()
15
16      # 1对1关系
17      stu1 = await Student.create(name='张飞')
18      # pro1 = await
19      StudentProfile.create(address='西域',
20      phone='123456')
21      # stu1.profile = pro1

```

```
19     # await stu1.save()
20
21     # pro2 = await
StudentProfile.create(address='东域',
phone='654321')
22     # stu2 = await Student.create(name='刘
备', profile=pro2)
23
24     # # 1对多关系
25     # grade1 = await Grade.create(score=100,
student=stu1)
26     # grade2 = await Grade.create(score=90,
student=stu1)
27     # grade3 = await Grade.create(score=80,
student=stu1) # 创建时直接关联
28
29     # 多对多
30     stu1 = await Student.create(name='张飞')
31     stu2 = await Student.create(name='刘备')
32     course1 = await
Course.create(name='Python')
33     course2 = await
Course.create(name='TypeScript')
34     from model24 import StudentCourse
35     await
StudentCourse.create(students=stu1,
courses=course1)
36     await
StudentCourse.create(students=stu1,
courses=course2)
```

```
37     await
StudentCourse.create(students=stu2,
courses=course1)
38     await
StudentCourse.create(students=stu2,
courses=course2)
39
40
41 async def update_data():
42     await init()
43     # stu = await
Student.filter(id=1).first()
44     # pro = await
StudentProfile.create(address='南域',
phone='123456')
45     # stu.profile = pro
46     # await stu.save()
47
48     # grade = await
Grade.filter(id=1).first()
49     stu = await Student.filter(id=2).first()
50     # grade.student = stu
51     # await grade.save()
52     await
Grade.filter(id=1).update(student=stu)
53
54
55 async def delete_data():
56     await init()
57     stu = await Student.filter(id=2).first()
58     await stu.delete()
59
```

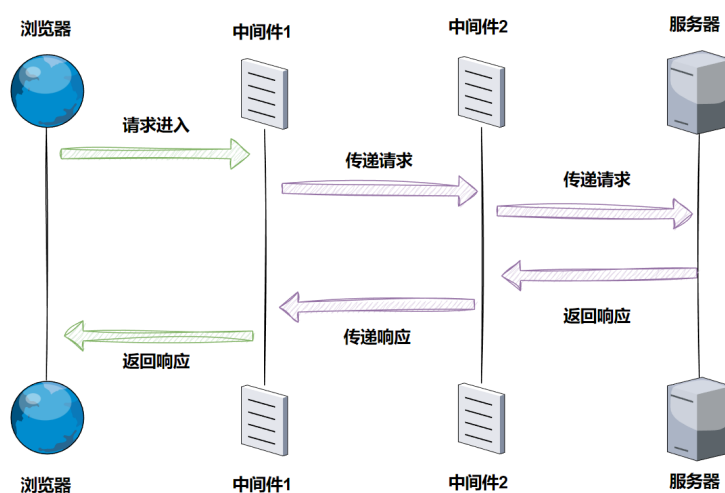
```
60
61 async def query_data():
62     await init()
63     stu = await
Student.get(id=3).prefetch_related("profile"
).first()
64     print(stu.profile.address)
65     print(stu.profile.phone)
66
67     stu2 = await Student.get(id=3)
68     pro = await stu2.profile
69     print(pro.address)
70     print(pro.phone)
71
72     stu3 = await
Student.get(id=3).prefetch_related("grades")
.all()
73     for grade in stu3.grades:
74         print(grade.score)
75
76 if __name__ == '__main__':
77     # run_async(create_data())
78     # run_async(update_data())
79     run_async(query_data())
80
```

FastAPI中间件



中间件是位于客户端和应用程序核心逻辑之间的软件层，用于：

- 拦截请求和响应
- 在请求到达路由处理程序之前进行处理
- 在响应返回给客户端之前进行处理



基本结构

```
1 @app.middleware("http")
2 async def custom_middleware(request:
  Request, call_next):
3     # 请求前处理
4     start_time = time.time()
5
6     # 调用后续中间件或路由处理
7     response = await call_next(request)
8
9     # 响应后处理
10    process_time = time.time() - start_time
11    response.headers["X-Process-Time"] =
  str(process_time)
12
13    return response
```

关键组件

- **request: Request** : 传入的请求对象
- **call_next** : 调用链中的下一个中间件/路由
- **返回值**: 必须返回 **Response** 对象

请求日志记录

```
1 @app.middleware("http")
2 async def log_requests(request: Request,
  call_next):
3     logger.info(f"Request: {request.method}
  {request.url}")
4     response = await call_next(request)
5     logger.info(f"Response:
  {response.status_code}")
6     return response
```

实时效果反馈

1. 在 FastAPI 中，如何正确添加一个自定义中间件？

- A 使用 `@app.middleware("http")` 装饰器
- B 使用 `@app.route()` 装饰器
- C 直接在路由函数中定义中间件逻辑
- D 使用 `@app.exception_handler` 装饰器

答案

1=>A

FastAPI跨域资源共享



同源策略 (SOP)

同源策略是浏览器的一种安全机制，限制了一个源 (origin) 的网页如何与另一个源的资源进行交互。

- 源 (origin) 由**协议** (如HTTP/HTTPS)、**域名** (如example.com) 和**端口** (如80或443) 组成。
- 例如，<https://example.com:443> 和 <http://example.com:80> 是不同源，因为协议和端口不同。

同源策略防止恶意网站通过脚本（如JavaScript）未经授权访问其他网站的数据，例如窃取用户的敏感信息

但是：现代Web应用经常需要跨源请求！

CORS的工作原理

CORS（Cross-Origin Resource Sharing，跨源资源共享）是一种基于HTTP的机制，它允许服务器指示哪些其他源（域名、协议或端口）可以访问其资源，从而绕过浏览器的同源策略（Same-Origin Policy, SOP）限制

示例

测试代码

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>CORS Test</title>
5 </head>
6 <body>
7   <h1>CORS 测试</h1>
8   <button onclick="testCors()">测试
CORS</button>
9   <p id="result">这块儿显示响应</p>
10  <script>
11    async function testCors() {
12      try {
13        const response = await
fetch('http://127.0.0.1:8000/info', {
          method: 'GET',
```

```

15         headers: {
16             'Content-Type':
17                 'application/json'
18         }
19     });
20     const data = await response.json();
21
22     document.getElementById('result').textContent = 'Success: ' + JSON.stringify(data);
23 } catch (error) {
24     document.getElementById('result').textContent = 'Error: ' + error.message;
25 }
26 </script>
27 </body>
28 </html>

```

自己实现

```

1  # CORS使用
2  from fastapi import FastAPI
3  from fastapi.responses import Response
4
5  app = FastAPI()
6
7
8  @app.middleware('http')
9  async def middleware(request, call_next):
10     # 处理 OPTIONS 预检请求

```

```

11     if request.method == "OPTIONS":
12         headers = {
13             "Access-Control-Allow-Origin":
14             "*",
15             "Access-Control-Allow-Methods":
16             "GET, POST, PUT, DELETE, OPTIONS",
17             "Access-Control-Allow-Headers":
18             "Content-Type, Authorization"
19         }
20         return Response(status_code=200,
21             headers=headers)
22     response = await call_next(request)
23     response.headers['Access-Control-Allow-
24     Origin'] = '*'
25     return response
26
27
28 @app.get('/info')
29 async def get_info():
30     return '内容成功获取!'
31
32
33 if __name__ == "__main__":
34     import uvicorn
35     uvicorn.run('main:app',
36         host="127.0.0.1", port=8000, reload=True)
37
38
39

```

使用官方提供中间件

```

1 # CORS使用
2 from fastapi import FastAPI

```

```
3 from fastapi.middleware.cors import
  CORSMiddleware
4
5 app = FastAPI()
6
7 # 添加 CORS 中间件
8 app.add_middleware(
9     CORSMiddleware,
10     allow_origins=["*"], # 或指定前端域名, 如
    ["http://localhost:8080"]
11     allow_credentials=True, # 允许携带 Cookie
12     allow_methods=["*"], # 允许所有方法, 包括
    OPTIONS
13     allow_headers=["*"], # 允许所有头部
14 )
15
16
17 @app.get('/info')
18 async def get_info():
19     return '内容成功获取!'
20
21
22 if __name__ == "__main__":
23     import uvicorn
24     uvicorn.run('main:app',
        host="127.0.0.1", port=8000, reload=True)
```

实时效果反馈

1. CORS（跨源资源共享）的主要目的是什么？

- A 完全禁止所有跨源请求以确保安全性
- B 允许服务器控制哪些跨源请求可以访问其资源

- C** 强制所有Web请求使用HTTPS协议
- D** 提高浏览器的页面加载速度

答案

1=>B

FastAPI业务划分



APIRouter 核心作用

- ① **模块化架构**：将大型应用拆分为独立功能模块
- ② **路由分组**：统一管理相关端点
- ③ **组织优化**：解耦业务逻辑，提升可维护性
- ④ 等等

基础使用示例

```
1 # 创建路由器
2 from fastapi import APIRouter
3
4 router = APIRouter()
5
6 # 添加路由
7 @router.get("/items/")
8 async def read_items():
9     return [{"name": "Item A"}, {"name":
10 "Item B"}]
11
12 @router.post("/items/")
13 async def create_item(item: dict):
14     return {"status": "created", "item":
15 item}
```

多路由器嵌套

```
1 # 主路由器
2 main_router = APIRouter()
3
4 # 子路由器
5 admin_router = APIRouter(prefix="/admin")
6 user_router = APIRouter(prefix="/users")
7
8 @admin_router.post("/dashboard")
9 async def admin_dashboard(): ...
10
11 @user_router.get("/profile")
12 async def user_profile(): ...
13
14 # 嵌套注册
15 main_router.include_router(admin_router)
```

实时效果反馈

1. APIRouter的主要作用是什么?

- A** 替代FastAPI主应用实例
- B** 实现数据库连接池
- C** 模块化组织路由端点
- D** 生成前端用户界面

2. 如何为路由器下所有路由添加 `/api/v1` 前缀,比较好?

- A** 在每个路由装饰器手动添加前缀
- B** 使用 `APIRouter(prefix="/api/v1")`
- C** 修改FastAPI应用的mount路径
- D** 在中间件中重写请求路径

答案

1=>C 2=>B

FastAPI项目结构



FastAPI 项目中，代码全部集中在单个文件中，虽然这种方式**适合小型项目或原型开发**，但在企业级应用中，将代码拆分为**多个文件和目录**是更优的实践，主要原因：

- **职责分离：**

将应用的各个组成部分（如数据库配置、模型、模式、路由、中间件）分别放置在单独的文件或目录中，使代码结构更清晰，便于理解和维护

- **提高代码可读性和可维护性：**

将代码拆分为小而专一的文件，开发者可以快速定位和修改特定功能，而无需在单一长文件中搜索

- **增强可扩展性：**

模块化的结构便于扩展，当需要添加新功能（如新的模型或路由）时，只需在相应的目录中创建新文件，而不影响现有代码

- **清晰的依赖管理：**

通过在每个文件中显式导入依赖，可以清楚地看到模块之间的关系，便于调试和理解代码逻辑

- **便于团队协作：**

在企业开发中，多个开发者可能同时开发不同功能，拆分后的文件结构允许团队成员并行工作。例如，一个开发者修改用户路由，另一个开发者可以同时处理商品路由，互不干扰。

- **符合企业级项目规范：**

企业级项目通常需要遵循标准化的代码组织规范，拆分文件是行业最佳实践之一。这种结构也便于集成自动化测试、CI/CD 流程和代码审查。

项目结构示例

按软件功能目录结构示例

```

1 project/
2   └─ main.py                # 主入口文件，初始
   化 FastAPI 应用并集成路由、中间件
3   └─ config/
4       └─ database.py        # 数据库配置和
   Tortoise-ORM 初始化
5   └─ models/
6       └─ user.py            # 数据模型定义（如
   User 模型）
7   └─ schemas/
8       └─ user.py            # Pydantic 模式定
   义，用于输入输出验证
9   └─ routers/
10      └─ __init__.py         # 标记 routers 为
   Python 包
11      └─ user.py             # 用户相关的 API
   路由
12      └─ item.py             # 商品相关的 API
   路由
13  └─ middleware/
14      └─ user_middleware.py  # 自定义中间
   件逻辑

```

按业务模块的目录结构示例

```

1 project/

```

```

2 |— main.py                                # 主入口文件，初始
   化 FastAPI 应用
3 |— config/
4 |   |— database.py                        # 数据库配置和
   Tortoise-ORM 初始化
5 |— modules/                              # 业务模块目录
6 |   |— user/                             # 用户管理模块
7 |       |— __init__.py                  # 标记 user 为
   Python 包
8 |       |— models.py                   # 用户相关的数据模型
9 |       |— schemas.py                  # 用户相关的
   Pydantic 模式
10 |       |— routers.py                 # 用户相关的 API 路
   由
11 |       |— item/                      # 商品管理模块
12 |           |— __init__.py           # 标记 item 为
   Python 包
13 |           |— models.py              # 商品相关的数据模型
   （当前为空）
14 |           |— schemas.py             # 商品相关的
   Pydantic 模式（当前为空）
15 |           |— routers.py            # 商品相关的 API 路
   由
16 |— middleware/
17 |   |— user_middleware.py              # 自定义中间
   件逻辑

```

